

IS Lab 6

Genetic Algorithm for 0/1 Knapsack

IT24610796

Problem

Given n items with weights $w[i]$ and profits $p[i]$, select a subset maximizing total profit with total weight $\leq$ capacity C . NP-hard; solved optimally via DP for small instances.

Instance

Dataset: OR-Library P07 (15 items, capacity 750)

$n=15$, $C=750$

Optimal (DP): 1458

GA Design

Binary encoding: 1 = take item. Fitness: hard penalty = profit - $1000 \cdot \max(0, \text{weight} - \text{capacity})$. Selection: tournament $k=3$. Crossover: 2-point ($p_c=0.9$). Mutation: bit-flip ($p_m=0.02$). Elitism: 2. Pop: 150, Gens: 200. Seed: 42.

Results

Baseline ($k=3$, 2pt, $p_m=0.02$): optimal=1458, GA=1444, gap=0.96%.

Tournament $k=2$: 1442 (99.8% of baseline) - less pressure, worse.

Tournament $k=4$: 1450 (100.4% of baseline) - best variant.

Roulette: infeasible due to negative fitness values in DEAP.

Roulette+Shift: 1442 (99.8%). Rank: 1440 (99.7%).

1-point crossover: 1440 (99.7%). 2-point: 1444 (100%).

Uniform crossover: 1436 (99.4%) - too much disruption.

$p_m=0.01$: 1440 (99.7%). $p_m=0.02$: 1444 (100%). $p_m=0.10$: 1428 (99.0%).

Convergence

Best fitness rises from 500 to 1444 over 200 generations. Average rises from 400 to 1188. Gap between best and avg widens over time (selection pressure). Plateau reached around gen 100-120.

Conclusion

GA achieves 99% of optimal for this medium instance. Tournament $k=4$ gives best results due to higher selection pressure on a small population. 2-point crossover preserves building blocks better than uniform. $p_m=0.02$ balances exploration vs exploitation. Convergence plateaus as population converges.

Code Repository

<https://github.com/IT24610796/lab>

Dependencies: Python 3.11, DEAP, matplotlib. Run: `python ga_knapsack.py` (from project root). Virtual env: `.venv/` with packages in `requirements.txt`.